CICLO ESCOLAR 2020-2021

PLANTEL:

SEMESTRE: FEBRERO - JULIO 2021

CECyTE Campeche

CUADERNILLO DE TRABAJO DEL MÓDULO III SUBMÓDULO II

NOMBRE DEL ALUMNO:

_____

SEDUC

SEGUNDO PARCIAL

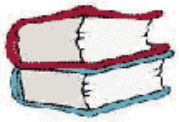
APRENDIZAJE ESPERADO 1	
	Competencia profesional: • Utiliza JavaScript para el manejo de eventos.
	Situación: • Por medio de algoritmos.
	Conceptos fundamentales: • Estructuras de control de flujo **Conceptos subsidiarios:** • Bucles • While • Do…While • For • For… in
	Producto Esperado: • Resumen.
	Instrumento de evaluación: • Lista de cotejo EP1-1
	Ponderación: • **30%**

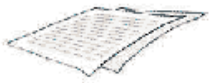
Producto: Resumen

¿Qué es un resumen?

El resumen es la exposición breve de lo esencial de un tema o materia, tanto efectuada de manera oral como escrita. Consiste en reducir o sintetizar el contenido de una lectura, texto, documento. El resumen es un texto que contiene la información más importante de un texto más largo. Su función principal es informar y para esto se sintetizan las ideas principales y se quita todo aquello que sobra

texto base → resumen





Características y función del resumen

- *El resumen contiene todas las ideas básicas del texto original*
- *Presenta ideas relacionadas por enlaces como conjunciones, preposiciones o signos*
- *Se escribe con estilo propio sin influencias del estilo del autor original*
- *Es un texto más breve que el original*
- *Respeta las normas básicas de ortografía y redacción*

Pasos para hacer un resumen

1.- Lee con atención el texto, de preferencia dos o más veces para que detectes las ideas principales. Busca el significado de los términos que no conozcas. Recuerda que la comprensión completa del texto es fundamental para hacer un buen resumen.

2.- Identifica las ideas principales del texto. Puedes subrayarlas, hacer una esquema o tomar notas en una libreta.

3.- Redacta el resumen construyendo una secuencia de ideas generales y omite todo lo que no sea importante.

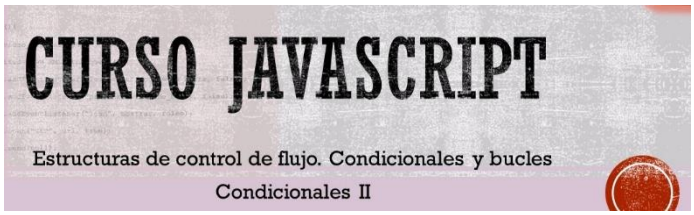
4.- Adecúa el lenguaje usado para expresar fielmente las ideas del autor original

5.- Corrige el texto para que tenga coherencia. Asegúrate que contenga todas las ideas importantes y que se conserve el estilo.

¡Empezemos!



ESTRUCTURAS DE CONTROL DE FLUJO.



Los programas que se pueden realizar utilizando solamente variables y operadores son una simple sucesión lineal de instrucciones básicas.

Sin embargo, no se pueden realizar programas que muestren un mensaje si el valor de una variable es igual a un valor determinado y no muestren el mensaje en el resto de casos. Tampoco se puede repetir de forma eficiente una misma instrucción, como por ejemplo sumar un determinado valor a todos los elementos de un array.

Para realizar este tipo de programas son necesarias las **estructuras de control de flujo**, que son instrucciones del tipo *"si se cumple esta condición, hazlo; si no se cumple, haz esto otro"*. También existen instrucciones del tipo *"repite esto mientras se cumpla esta condición"*.

Si se utilizan estructuras de control de flujo, los programas dejan de ser una sucesión lineal de instrucciones para convertirse en programas *inteligentes* que pueden tomar decisiones en función del valor de las variables.

BUCLES

Entendamos por bucle un conjunto de sentencias que se repite un número determinado de veces. Supongamos que por ejemplo queremos tomar el contenido de un array y sumarle una cantidad. Con la estructura if sólo conseguimos que lo que está dentro de las llaves se ejecute una sola vez, por lo que no sería válida para lo que perseguimos.
Para este tipo de necesidades tenemos las estructuras llamadas bucles. En ellos se establece una condición (igual que lo hacíamos en el if) que se evaluará para ver si se cumpla o no. Ésta será la condición de entrada al bucle. Las instrucciones interiores, encerradas entre llaves, se realizan mientras se siga cumpliendo la condición especificada en la entrada del bucle.

Estructura while

La estructura while forma un bucle en el cual el código indicado se repite mientras se cumpla una determinada condición. Su sintaxis es:

```
while( /*condicion*/ ) {
//Código que se repite
}
```

Escribimos en primer lugar la palabra reservada while seguida de la condición entre paréntesis. Después entre llaves ponemos las sentencias que deben repetirse. Para que esta estructura funcione correctamente dentro de la condición debe estar la variable o el elemento que controla el bucle, el cual debe estar definido anteriormente. En el código que se repite debe cambiar en cada vuelta la variable o elemento de control, de manera que haya un momento en el cual la condición no se cumpla.

Estructura do while

Como una variante de la estructura **while** tenemos la etructura **do while** con la siguiente sintaxis:

```
do { //Código que se repite
}
while( /*condicion*/ )
```

Aquí, aparte de usar la palabra reservada do, la condición se indica al final. Esto hace que el código que se indica dentro de la estructura se ejecuta siempre al menos una vez, aunque la condición no se cumpla. Por lo demás funciona igual que la estructura while.

Estructura for

Las estructuras if y if...else no son muy eficientes cuando se desea ejecutar de forma repetitiva una instrucción. Por ejemplo, si se quiere mostrar un mensaje cinco veces, se podría pensar en utilizar el siguiente if:

```
var veces = 0;
if(veces < 4) {
alert("Mensaje");
veces++;
}
```

Se comprueba si la variable veces es menor que 4. Si se cumple, se entra dentro del if(), se muestra el mensaje y se incrementa el valor de la variable veces. Así se debería seguir ejecutando hasta mostrar el mensaje las cinco veces deseadas.

Sin embargo, el funcionamiento real del script anterior es muy diferente al deseado, ya que solamente se muestra una vez el mensaje por pantalla. La razón es que la ejecución de la estructura if() no se repite y la comprobación de la condición sólo se realiza una vez, independientemente de que dentro del if() se modifique el valor de la variable utilizada en la condición.

La estructura for permite realizar este tipo de repeticiones (también llamadas bucles) de una forma muy sencilla. No obstante, su definición formal no es tan sencilla como la de if():

```
for(inicializacion; condicion; actualizacion) {
...
}
```

La idea del funcionamiento de un bucle for es la siguiente: *"mientras la condición indicada se siga cumpliendo, repite la ejecución de las instrucciones definidas dentro del for. Además, después de cada repetición, actualiza el valor de las variables que se utilizan en la condición"*.

- La "inicialización" es la zona en la que se establece los valores iniciales de las variables que controlan la repetición.
- La "condición" es el único elemento que decide si continua o se detiene la repetición.
- La "actualización" es el nuevo valor que se asigna después de cada repetición a las variables que controlan la repetición.

```
var mensaje = "Hola, estoy dentro de un bucle";
for(var i = 0; i < 5; i++) {
alert(mensaje);
}
```

La parte de la inicialización del bucle consiste en:
`var i = 0;`
Por tanto, en primer lugar se crea la variable i y se le asigna el valor de 0. Esta zona de inicialización solamente se tiene en consideración justo antes de comenzar a ejecutar el bucle.
Las siguientes repeticiones no tienen en cuenta esta parte de inicialización.
La zona de condición del bucle es:

i < 5
Los bucles se siguen ejecutando mientras se cumplan las condiciones y se dejan de ejecutar justo después de comprobar que la condición no se cumple. En este caso, mientras la variable i valga menos de 5 el bucle se ejecuta indefinidamente.
Como la variable i se ha inicializado a un valor de 0 y la condición para salir del bucle es que i sea menor que 5, si no se modifica el valor de i de alguna forma, el bucle se repetiría indefinidamente.
Como la variable i se ha inicializado a un valor de 0 y la condición para salir del bucle es que i sea menor que 5, si no se modifica el valor de i de alguna forma, el bucle se repetiría indefinidamente. Por ese motivo, es imprescindible indicar la zona de actualización, en la que se modifica el valor de las variables que controlan el bucle:
i++
En este caso, el valor de la variable i se incrementa en una unidad después de cada repetición. La zona de actualización se ejecuta después de la ejecución de las instrucciones que incluye el for. Así, durante la ejecución de la quinta repetición el valor de i será 4. Después de la quinta ejecución, se actualiza el valor de i, que ahora valdrá 5. Como la condición es que i sea menor que 5, la condición ya no se cumple y las instrucciones del for no se ejecutan una sexta vez.

Normalmente, la variable que controla los bucles for se llama i, ya que recuerda a la palabra índice y su nombre tan corto ahorra mucho tiempo y espacio.
Un ejemplo anterior que mostraba los días de la semana contenidos en un array se puede rehacer de forma más sencilla utilizando la estructura for:

```
var dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"];
for(var i=0; i<7; i++) {
alert(dias[i]);
}
```

Estructura for...in

Una estructura de control derivada de for es la estructura for...in. Su definición exacta implica el uso de objetos, que es un elemento de programación avanzada que no se va a estudiar. Por tanto, solamente se va a presentar la estructura for...in adaptada a su uso en arrays. Su definición formal adaptada a los arrays es:

```
for(indice in array) {
...
}
```

Si se quieren recorrer todos los elementos que forman un array, la estructura for...in es la forma más eficiente de hacerlo, como se muestra en el siguiente ejemplo:

```
var dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"];
for(i in dias) {
alert(dias[i]);
}
```

La variable que se indica como indice es la que se puede utilizar dentro del bucle for...in para acceder a los elementos del array. De esta forma, en la primera repetición del bucle la variable i vale 0 y en la última vale 6.

Esta estructura de control es la más adecuada para recorrer arrays (y objetos), ya que evita tener que indicar la inicialización y las condiciones del bucle for simple y funciona correctamente cualquiera que sea la longitud del array. De hecho, sigue funcionando igual aunque varíe el número de elementos del array.

1. **Evidencia N°1.** ***Elabora en el siguiente recuadro un resumen de las estructuras de control de flujo.***

APRENDIZAJE ESPERADO 2	
	Competencia profesional: • Utiliza JavaScript para el manejo de eventos.
	Situación:
	Conceptos fundamentales: • Funciones y objetos **Conceptos subsidiarios:** • Definición de funciones • Argumentos y variables de retorno. • Ámbito de las variables • Creación de objetos
	Producto Esperado: • Mapa Conceptual.
	Instrumento de evaluación: • Lista de cotejo EP1-1
	Ponderación: • 30%

Funciones y objetos.

Las estructuras de control, los operadores y todas las utilidades propias de JavaScript anteriores, permiten crear scripts sencillos y de mediana complejidad. Sin embargo, para las aplicaciones más complejas son necesarios otros elementos como las funciones y otras estructuras de control más avanzadas.

Funciones

Cuando se desarrolla una aplicación compleja, es muy habitual utilizar una y otra vez las mismas instrucciones. Un script para una tienda de comercio electrónico por ejemplo, tiene que calcular el precio total de los productos varias veces, para añadir los impuestos y los gastos de envío. Cuando una serie de instrucciones se repiten una y otra vez, se complica demasiado el código fuente de la aplicación, ya que:

▪ El código de la aplicación es mucho más largo porque muchas instrucciones están repetidas.

▪ Si se quiere modificar alguna de las instrucciones repetidas, se deben hacer tantas modificaciones como veces se haya escrito esa instrucción, lo que se convierte en un trabajo muy pesado y muy propenso a cometer errores.

Las funciones son la solución a todos estos problemas, tanto en JavaScript como en el resto de lenguajes de programación. Una función es un conjunto de instrucciones que se agrupan para realizar una tarea concreta y que se pueden reutilizar fácilmente.
En el siguiente ejemplo, las instrucciones que suman los dos números y muestran un mensaje con el resultado se repiten una y otra vez:

```
var resultado;
var numero1 = 3;
var numero2 = 5;
// Se suman los números y se muestra el resultado
resultado = numero1 + numero2;
alert("El resultado es " + resultado);
numero1 = 10;
numero2 = 7;
// Se suman los números y se muestra el resultado
resultado = numero1 + numero2;
alert("El resultado es " + resultado);
numero1 = 5;
numero2 = 8;
// Se suman los números y se muestra el resultado
resultado = numero1 + numero2;
alert("El resultado es " + resultado);
...
```

Aunque es un ejemplo muy sencillo, parece evidente que repetir las mismas instrucciones a lo largo de todo el código no es algo recomendable. La solución que proponen las funciones consiste en extraer las instrucciones que se repiten y sustituirlas por una instrucción del tipo *"en este punto, se ejecutan las instrucciones que se han extraído"*:

```
var resultado;
var numero1 = 3;
var numero2 = 5;
/* En este punto, se llama a la función que suma
2 números y muestra el resultado */
numero1 = 10;
numero2 = 7;
/* En este punto, se llama a la función que suma
2 números y muestra el resultado */
numero1 = 5;
numero2 = 8;
/* En este punto, se llama a la función que suma
2 números y muestra el resultado */
...
```

Para que la solución del ejemplo anterior sea válida, las instrucciones comunes se tienen que agrupar en una función a la que se le puedan indicar los números que debe sumar antes de

mostrar el mensaje.
Por lo tanto, en primer lugar se debe crear la función básica con las instrucciones comunes. Las funciones en JavaScript se definen mediante la palabra reservada function, seguida del nombre de la función. Su definición formal es la siguiente:

```
function nombre_funcion() {
...
}
```

El nombre de la función se utiliza para *llamar* a esa función cuando sea necesario. El concepto es el mismo que con las variables, a las que se les asigna un nombre único para poder utilizarlas dentro del código. Después del nombre de la función, se incluyen dos paréntesis cuyo significado se detalla más adelante. Por último, los símbolos { y } se utilizan para encerrar todas las instrucciones que pertenecen a la función (de forma similar a como se encierran las instrucciones en las estructuras if o for).

Volviendo al ejemplo anterior, se crea una función llamada suma_y_muestra de la siguiente forma:

```
 function suma_y_muestra() {
resultado = numero1 + numero2;
alert("El resultado es " + resultado);
}
```

Aunque la función anterior está correctamente creada, no funciona como debería ya que le faltan los "argumentos", que se explican en la siguiente sección. Una vez creada la función, desde cualquier punto del código se puede llamar a la función para que se ejecuten sus instrucciones (además de "llamar a la función", también se suele utilizar la expresión "invocar a la función"). La llamada a la función se realiza simplemente indicando su nombre, incluyendo los paréntesis del final y el carácter ; para terminar la instrucción:

```
function suma_y_muestra() {
resultado = numero1 + numero2;
alert("El resultado es " + resultado);
}
var resultado;
var numero1 = 3;
var numero2 = 5;
suma_y_muestra();
numero1 = 10;
numero2 = 7;
suma_y_muestra();
```

```
numero1 = 5;
numero2 = 8;
suma_y_muestra();
...
```

El código del ejemplo anterior es mucho más eficiente que el primer código que se mostró, ya que no existen instrucciones repetidas. Las instrucciones que suman y muestran mensajes se han agrupado bajo una función, lo que permite ejecutarlas en cualquier punto del programa simplemente indicando el nombre de la función.
Lo único que le falta al ejemplo anterior para funcionar correctamente es poder indicar a la función los números que debe sumar. Cuando se necesitan pasar datos a una función, se utilizan los *"argumentos"*.

Argumentos y valores de retorno

Las funciones más sencillas no necesitan ninguna información para producir sus resultados. Sin embargo, la mayoría de funciones de las aplicaciones reales deben acceder al valor de algunas variables para producir sus resultados.
Las variables que necesitan las funciones se llaman *argumentos*. Antes de que pueda utilizarlos, la función debe indicar cuántos argumentos necesita y cuál es el nombre de cada argumento. Además, al invocar la función, se deben incluir los valores que se le van a pasar a la función. Los argumentos se indican dentro de los paréntesis que van detrás del nombre de la función y se separan con una coma (,).
Siguiendo el ejemplo anterior, la función debe indicar que necesita dos argumentos, correspondientes a los dos números que tiene que sumar:

```
function suma_y_muestra(primerNumero, segundoNumero) { ... }
```

A continuación, para utilizar el valor de los argumentos dentro de la función, se debe emplear el mismo nombre con el que se definieron los argumentos:

```
function suma_y_muestra (primerNumero, segundoNumero) { ... }
var resultado = primerNumero + segundoNumero;
alert("El resultado es " + resultado);
}
```

Dentro de la función, el valor de la variable primerNumero será igual al primer valor que se le pase a la función y el valor de la variable segundoNumero será igual al segundo valor que se le pasa. Para pasar valores a la función, se incluyen dentro de los paréntesis utilizados al llamar a la función:

```
// Definición de la función
function suma_y_muestra(primerNumero, segundoNumero) { ... }
var resultado = primerNumero + segundoNumero;
alert("El resultado es " + resultado);
}

// Declaración de las variables
var numero1 = 3;
var numero2 = 5;
// Llamada a la función
suma_y_muestra(numero1, numero2);
```

En el código anterior, se debe tener en cuenta que:

▪ Aunque casi siempre se utilizan variables para pasar los datos a la función, se podría haber utilizado directamente el valor de esas variables: suma_y_muestra(3, 5);

▪ El número de argumentos que se pasa a una función debería ser el mismo que el número de argumentos que ha indicado la función. No obstante, JavaScript no muestra ningún error si se pasan más o menos argumentos de los necesarios.

▪ El orden de los argumentos es fundamental, ya que el primer dato que se indica en la llamada, será el primer valor que espera la función; el segundo valor indicado en la llamada, es el segundo valor que espera la función y así sucesivamente.

▪ Se puede utilizar un número ilimitado de argumentos, aunque si su número es muy grande, se complica en exceso la llamada a la función.

▪ No es obligatorio que coincida el nombre de los argumentos que utiliza la función y el nombre de los argumentos que se le pasan. En el ejemplo anterior, los argumentos que se pasan son numero1 y numero2 y los argumentos que utiliza la función son primerNumero y segundoNumero.

A continuación, se muestra otro ejemplo de una función que calcula el precio total de un producto a partir de su precio básico:

```
// Definición de la función
function calculaPrecioTotal(precio) {
```

```
var impuestos = 1.16;
var gastosEnvio = 10;
var precioTotal = ( precio * impuestos ) + gastosEnvio;
}
// Llamada a la función
calculaPrecioTotal(23.34);
```

La función anterior toma como argumento una variable llamada precio y le suma los impuestos y los gastos de envío para obtener el precio total. Al llamar a la función, se pasa directamente el valor del precio básico mediante el número 23.34.

No obstante, el código anterior no es demasiado útil, ya que lo ideal sería que la función pudiera devolver el resultado obtenido para guardarlo en otra variable y poder seguir trabajando con este precio total:

```
function calculaPrecioTotal(precio) {
var impuestos = 1.16;
var gastosEnvio = 10;
var precioTotal = ( precio * impuestos ) + gastosEnvio;
}
// El valor devuelto por la función, se guarda en una variable
var precioTotal = calculaPrecioTotal (23.34);
// Seguir trabajando con la variable "precioTotal"
```

Afortunadamente, las funciones no solamente pueden recibir variables y datos, sino que también pueden devolver los valores que han calculado. Para devolver valores dentro de una función, se utiliza la palabra reservada return. Aunque las funciones pueden devolver valores de cualquier tipo, solamente pueden devolver un valor cada vez que se ejecutan.

```
function calculaPrecioTotal(precio) {
var impuestos = 1.16;
var gastosEnvio = 10;
var precioTotal = ( precio * impuestos ) + gastosEnvio;
return precioTotal;
}
var precioTotal = calculaPrecioTotal(23.34);
// Seguir trabajando con la variable "precioTotal"
```

Para que la función devuelva un valor, solamente es necesario escribir la palabra reservada return junto con el nombre de la variable que se quiere devolver. En el ejemplo anterior, la

ejecución de la función llega a la instrucción return precioTotal; y en ese momento, devuelve el valor que contenga la variable precioTotal.

Como la función devuelve un valor, en el punto en el que se realiza la llamada, debe indicarse el nombre de una variable en el que se guarda el valor devuelto:

```
var precioTotal = calculaPrecioTotal(23.34);
```

Si no se indica el nombre de ninguna variable, JavaScript no muestra ningún error y el valor devuelto por la función simplemente se pierde y, por tanto, no se utilizará en el resto del programa. En este caso, tampoco es obligatorio que el nombre de la variable devuelta por la función coincida con el nombre de la variable en la que se va a almacenar ese valor.

Si la función llega a una instrucción de tipo return, se devuelve el valor indicado y finaliza la ejecución de la función. Por tanto, todas las instrucciones que se incluyen después de un return se ignoran y por ese motivo la instrucción return suele ser la última de la mayoría de funciones.

Para que el ejemplo anterior sea más completo, se puede añadir otro argumento a la función que indique el porcentaje de impuestos que se debe añadir al precio del producto. Evidentemente, el nuevo argumento se debe añadir tanto a la definición de la función como a su llamada:

```
function calculaPrecioTotal(precio, porcentajeImpuestos) {
var gastosEnvio = 10;
var precioConImpuestos = (1 + porcentajeImpuestos/100) * precio;
var precioTotal = precioConImpuestos + gastosEnvio;
return precioTotal;
}
var precioTotal = calculaPrecioTotal(23.34, 16);
var otroPrecioTotal = calculaPrecioTotal(15.20, 4);
```

Para terminar de completar el ejercicio anterior, se puede redondear a dos decimales el precio total, devuelto por la función:

```
function calculaPrecioTotal(precio, porcentajeImpuestos) {
var gastosEnvio = 10;
var precioConImpuestos = (1 + porcentajeImpuestos/100) * precio;
var precioTotal = precioConImpuestos + gastosEnvio;
return precioTotal.toFixed(2);
}
var precioTotal = calculaPrecioTotal(23.34, 16);
```

Ámbitos de las variables.

El ámbito de una variable (llamado *"scope"* en inglés) es la zona del programa en la que se define la variable. JavaScript define dos ámbitos para las variables: global y local.

El siguiente ejemplo ilustra el comportamiento de los ámbitos:

```
function creaMensaje() {
var mensaje = “Mensaje de prueba”;
}
creaMensaje();
alert(mensaje);
```

El ejemplo anterior define en primer lugar una función llamada `creaMensaje` que crea una variable llamada mensaje. A continuación, se ejecuta la función mediante la llamada creaMensaje(); y seguidamente, se muestra mediante la función alert() el valor de una variable llamada mensaje.

Sin embargo, al ejecutar el código anterior no se muestra ningún mensaje por pantalla. La razón es que la variable mensaje se ha definido dentro de la función creaMensaje() y por tanto, es una **variable local** que solamente está definida dentro de la función.

Cualquier instrucción que se encuentre dentro de la función puede hacer uso de esa variable, pero todas las instrucciones que se encuentren en otras funciones o fuera de cualquier función no tendrán definida la variable mensaje. De esta forma, para mostrar el mensaje en el código anterior, la función alert() debe llamarse desde dentro de la función creaMensaje():

```
function creaMensaje() {
var mensaje = “Mensaje de prueba”;
alert(mensaje);
}
creaMensaje();
```

Además de variables locales, también existe el concepto de **variable global**, que está definida en cualquier punto del programa (incluso dentro de cualquier función).

```
var mensaje = “Mensaje de prueba”;
function muestraMensaje() {
alert(mensaje);
}
```

El código anterior es el ejemplo inverso al mostrado anteriormente. Dentro de la función muestraMensaje() se quiere hacer uso de una variable llamada mensaje y que no ha sido definida dentro de la propia función. Sin embargo, si se ejecuta el código anterior, sí que se muestra el mensaje definido por la variable mensaje.

El motivo es que en el código JavaScript anterior, la variable mensaje se ha definido fuera de cualquier función. Este tipo de variables automáticamente se transforman en variables globales y están disponibles en cualquier punto del programa (incluso dentro de cualquier función).

De esta forma, aunque en el interior de la función no se ha definido ninguna variable llamada mensaje, la variable global creada anteriormente permite que la instrucción alert() dentro de la función muestre el mensaje correctamente.

Si una variable se declara fuera de cualquier función, automáticamente se transforma en variable global independientemente de si se define utilizando la palabra reservada var o no. Sin embargo, las variables definidas dentro de una función pueden ser globales o locales.

Si en el interior de una función, las variables se declaran mediante var se consideran locales y las variables que no se han declarado mediante var, se transforman automáticamente en variables globales.

Por lo tanto, se puede rehacer el código del primer ejemplo para que muestre el mensaje correctamente. Para ello, simplemente se debe definir la variable dentro de la función sin la palabra reservada var, para que se transforme en una variable global:

```
function creaMensaje() {
mensaje = "Mensaje de prueba";
}
creaMensaje();
alert(mensaje);
```

¿Qué sucede si una función define una variable local con el mismo nombre que una variable global que ya existe? En este caso, las variables locales prevalecen sobre las globales, pero sólo dentro de la función:

```
var mensaje = "gana la de fuera";
function muestraMensaje() {
var mensaje = "gana la de dentro";
alert(mensaje);
}
alert(mensaje);
muestraMensaje();
alert(mensaje);
```

El código anterior muestra por pantalla los siguientes mensajes:

gana la de fuera
gana la de dentro
gana la de fuera

Dentro de la función, la variable local llamada mensaje tiene más prioridad que la variable global del mismo nombre, pero solamente dentro de la función.

¿Qué sucede si dentro de una función se define una variable global con el mismo nombre que otra variable global que ya existe? En este otro caso, la variable global definida dentro de la función simplemente modifica el valor de la variable global definida anteriormente:

```
var mensaje = "gana la de fuera";
function muestraMensaje() {
mensaje = "gana la de dentro";
alert(mensaje);
}
alert(mensaje);
muestraMensaje();
alert(mensaje);
```

En este caso, los mensajes mostrados son:

gana la de fuera
gana la de dentro
gana la de dentro

Evidencia N° 2. ***Elabora en el siguiente recuadro un mapa conceptual sobre las funciones y objetos en Javascrip.***

APRENDIZAJE ESPERADO 3	
	Competencia profesional: • Utiliza JavaScript para el manejo de eventos.
	Situación:
	Conceptos fundamentales: • Eventos **Conceptos subsidiarios:** • Eventos en JavaScript • Métodos de eventos disponibles en JavaScript • Aplicación de eventos en formularios
	Producto Esperado: • Ejercicios de eventos.
	Instrumento de evaluación: • Lista de cotejo EP1-1
	Ponderación: • 30%

Eventos.

Hasta ahora, todas las aplicaciones y scripts que se han creado tienen algo en común: se ejecutan desde la primera instrucción hasta la última de forma secuencial. Gracias a la estructura de control de flujo (if, for, while) es posible modificar ligeramente este comportamiento y repetir algunos trozos del script y saltarse otros trozos en función de algunas condiciones.

Este tipo de aplicaciones son poco útiles, ya que no interactúan con los usuarios y no pueden responder a los diferentes *eventos* que se producen durante la ejecución de una aplicación. Afortunadamente, las aplicaciones web creadas con el lenguaje JavaScript pueden utilizar el modelo de *programación basada en eventos*.

En este tipo de programación, los scripts se dedican a esperar a que el usuario *"haga algo"* (que pulse una tecla, que mueva el ratón, que cierre la ventana del navegador). A continuación, el script responde a la acción del usuario normalmente procesando esa información y generando un resultado.

Los eventos hacen posible que los usuarios transmitan información a los programas. JavaScript define numerosos eventos que permiten una interacción completa entre el usuario y las páginas/aplicaciones web. La pulsación de una tecla constituye un evento, así como pinchar o mover el ratón, seleccionar un elemento de un formulario, redimensionar la ventana del navegador, etc.

JavaScript permite asignar una función a cada uno de los eventos. De esta forma, cuando se produce cualquier evento, JavaScript ejecuta su función asociada. Este tipo de funciones se denominan *"event handlers"* en inglés y suelen traducirse por *"manejadores de eventos"*.

Modelos de eventos

Crear páginas y aplicaciones web siempre ha sido mucho más complejo de lo que debería serlo debido a las incompatibilidades entre navegadores. A pesar de que existen decenas de estándares para las tecnologías empleadas, los navegadores no los soportan completamente o incluso los ignoran.

Las principales incompatibilidades se producen en el lenguaje XHTML, en el soporte de hojas de estilos CSS y sobre todo, en la implementación de JavaScript. De todas ellas, la incompatibilidad más importante se da precisamente en el modelo de eventos del navegador. Así, existen hasta tres modelos diferentes para manejar los eventos dependiendo del navegador en el que se ejecute la aplicación.

6.1.1. Modelo básico de eventos

Este modelo simple de eventos se introdujo para la versión 4 del estándar HTML y se considera parte del nivel más básico de DOM. Aunque sus características son limitadas, es el único modelo que es compatible en todos los navegadores y por tanto, el único que permite crear aplicaciones que funcionan de la misma manera en todos los navegadores.

Modelo de eventos estándar

Las versiones más avanzadas del estándar DOM (DOM nivel 2) definen un modelo de eventos completamente nuevo y mucho más poderoso que el original. Todos los navegadores modernos lo incluyen, salvo Internet Explorer.

6.1.3. Modelo de eventos de Internet Explorer

Internet Explorer utiliza su propio modelo de eventos, que es similar pero incompatible con el modelo estándar. Se utilizó por primera vez en Internet Explorer 4 y Microsoft decidió seguir utilizándolo en el resto de versiones, a pesar de que la empresa había participado en la creación del estándar de DOM que define el modelo de eventos estándar.

Tipos de eventos

En este modelo, cada elemento o etiqueta XHTML define su propia lista de posibles eventos que se le pueden asignar. Un mismo tipo de evento (por ejemplo, pinchar el botón izquierdo del ratón) puede estar definido para varios elementos XHTML diferentes y un mismo elemento XHTML puede tener asociados varios eventos diferentes.

El nombre de cada evento se construye mediante el prefijo on, seguido del nombre en inglés de la acción asociada al evento. Así, el evento de pinchar un elemento con el ratón se denomina onclick y el evento asociado a la acción de mover el ratón se denomina onmousemove.

La siguiente tabla resume los eventos más importantes definidos por JavaScript:

Evento	Descripción	Elementos para los que está definido
onblur	Deseleccionar el elemento	<button>, <input>, <label>, <select>, <textarea>, <body>
onchange	Deseleccionar un elemento que se ha modificado	<input>, <select>, <textarea>
onclick	Pinchar y soltar el ratón	Todos los elementos
ondblclick	Pinchar dos veces seguidas con el ratón	Todos los elementos
onfocus	Seleccionar un elemento	<button>, <input>, <label>, <select>, <textarea>, <body>
onkeydown	Pulsar una tecla (sin soltar)	Elementos de formulario y <body>
onkeypress	Pulsar una tecla	Elementos de formulario y <body>
onkeyup	Soltar una tecla pulsada	Elementos de formulario y <body>
onload	La página se ha cargado completamente	<body>
onmousedown	Pulsar (sin soltar) un botón del ratón	Todos los elementos
onmousemove	Mover el ratón	Todos los elementos

Los eventos más utilizados en las aplicaciones web tradicionales son onload para esperar a que se cargue la página por completo, los eventos onclick, onmouseover, onmouseout para controlar el ratón y onsubmit para controlar el envío de los formularios.

Algunos eventos de la tabla anterior (onclick, onkeydown, onkeypress, onreset, onsubmit) permiten evitar la *"acción por defecto"* de ese evento. Más adelante se muestra en detalle este comportamiento, que puede resultar muy útil en algunas técnicas de programación.

Las acciones típicas que realiza un usuario en una página web pueden dar lugar a una sucesión de eventos. Al pulsar por ejemplo sobre un botón de tipo <input type="submit"> se desencadenan los eventos onmousedown, onclick, onmouseup y onsubmit de forma consecutiva.

Manejadores de eventos

Un evento de JavaScript por sí mismo carece de utilidad. Para que los eventos resulten útiles, se deben asociar funciones o código JavaScript a cada evento. De esta forma, cuando se produce un evento se ejecuta el código indicado, por lo que la aplicación puede *responder* ante cualquier evento que se produzca durante su ejecución.
Las funciones o código JavaScript que se definen para cada evento se denominan *"manejador de eventos"* y como JavaScript es un lenguaje muy flexible, existen varias formas diferentes de indicar los manejadores:

- Manejadores como atributos de los elementos XHTML.
- Manejadores como funciones JavaScript externas.
- Manejadores *"semánticos"*.

Evidencia N°3. ***Elabora en el siguiente recuadro el pseudocodigo de un programa que inclua manejadores de eventos en Javascrip.***

RUBRICA 1-1



COLEGIO DE ESTUDIOS CIENTÍFICOS Y TECNOLÓGICOS DEL ESTADO DE CAMPECHE
INSTRUMENTO DE EVALUACIÓN PARA LA ASIGNATURA DE PROGRAMACIÓN
MÓDULO II: APLICA METODOLOGÍAS DE DESARROLLO DE SOFTWARE CON HERRAMIENTAS DE PROGRAMACIÓN VISUAL
SUBMODULO I: APLICA LA METODOLOGÍA EN ESPIRAL CON PROGRAMACIÓN ORIENTADA A OBJETOS
REFORMA INTEGRAL DE LA EDUCACIÓN MEDIA SUPERIOR

LISTA DE COTEJO EP 1-1			
ALUMNO(A):			
PLANTEL: MIGUEL HIDALGO SEMESTRE: **CUARTO**	EVIDENCIA: **RESUMEN**		
CARRERA: **PROGRAMACIÓN**	ENTE EVALUADOR: ***HETEROEVALUACIÓN***		
SEGUNDO PARCIAL	TIPO DE EVALUACIÓN: ***FORMATIVA, SUMATIVA***		
COMPETENCIA PROFESIONAL: 1. Utiliza JavaScript para el manejo de eventos			
COMPETENCIA DISCIPLINAR: C12 Utiliza las tecnologías de la información y comunicación para investigar, resolver problemas, producir materiales y transmitir información.			
COMPETENCIA GENÉRICA: 4. Escucha e interpreta y emite mensaje pertinentes en distintos contextos mediante la utilizacion de medios, códigos y herramientas apropiadas.			
CRITERIOS A EVALUAR	**PUNTOS**		**OBSERVACIONES**
	VALOR	**LOGRADO**	
El resumen contiene:			
1. Portada. 2. Índice. 3. Desarrollo del tema. 3.4 Estructuras de control de flujo 3.4.2. Bucles 3.4.2.1. While 3.4.2.2. Do…While 3.4.2.3. For 3.4.2.4. For… in 4.- Conclusión 5.- Fuentes consultadas (2 o mas fuentes)	3 3 8 4 4		

6. Características: (-Orden en las ideas - Claridad - Concisión) 7.- Uso de las TIC para la búsqueda de información.	4 4		
TOTAL	**30**		
RESULTADO DE EVALUACIÓN:			
FECHA DE APLICACIÓN:			

RESPONSABLE DE LA EVALUACIÓN

COLEGIO DE ESTUDIOS CIENTÍFICOS Y TECNOLÓGICOS DEL ESTADO DE CAMPECHE
INSTRUMENTO DE EVALUACIÓN PARA LA ASIGNATURA DE PROGRAMACIÓN
MÓDULO II: APLICA METODOLOGÍAS DE DESARROLLO DE SOFTWARE CON HERRAMIENTAS DE PROGRAMACIÓN VISUAL
SUBMODULO I: APLICA LA METODOLOGÍA EN ESPIRAL CON PROGRAMACIÓN ORIENTADA A OBJETOS
REFORMA INTEGRAL DE LA EDUCACIÓN MEDIA SUPERIOR

LISTA DE COTEJO EP 1-1			
ALUMNO(A):			
PLANTEL: MIGUEL HIDALGO SEMESTRE: **CUARTO**	EVIDENCIA: **MAPA CONCEPTUAL**		
CARRERA: **PROGRAMACIÓN**	ENTE EVALUADOR: ***HETEROEVALUACIÓN***		
SEGUNDO PARCIAL	TIPO DE EVALUACIÓN: ***FORMATIVA, SUMATIVA***		
COMPETENCIA PROFESIONAL: 1. Utiliza JavaScript para el manejo de eventos			
COMPETENCIA DISCIPLINAR: C12 Utiliza las tecnologías de la información y comunicación para investigar, resolver problemas, producir materiales y transmitir información.			
COMPETENCIA GENÉRICA: 4. Escucha e interpreta y emite mensaje pertinentes en distintos contextos mediante la utilización de medios, códigos y herramientas apropiadas.			
CRITERIOS A EVALUAR	**PUNTOS**		**OBSERVACIONES**
	VALOR	**LOGRADO**	
El mapa conceptual contiene:			
1. El concepto principal es adecuado y pertinente con el tema. 2. El mapa conceptual incluye todos los conceptos importantes del tema. 3. La mayor parte de las preposiciones son válidas. 4. El mapa conceptual integra enlaces creativos y novedosos. 5. Los conceptos están ordenados jerárquicamente y presenta más de 3 niveles. 6. Entrega en tiempo y forma.	5 5 5 5 5 5		
TOTAL	**30**		
RESULTADO DE EVALUACIÓN:			
FECHA DE APLICACIÓN:			

RESPONSABLE DE LA EVALUACIÓN

COLEGIO DE ESTUDIOS CIENTÍFICOS Y TECNOLÓGICOS DEL ESTADO DE CAMPECHE

INSTRUMENTO DE EVALUACIÓN PARA LA ASIGNATURA DE PROGRAMACIÓN

MÓDULO II: APLICA METODOLOGÍAS DE DESARROLLO DE SOFTWARE CON HERRAMIENTAS DE PROGRAMACIÓN VISUAL

SUBMODULO I: APLICA LA METODOLOGÍA EN ESPIRAL CON PROGRAMACIÓN ORIENTADA A OBJETOS

REFORMA INTEGRAL DE LA EDUCACIÓN MEDIA SUPERIOR

<table>
<tr><td colspan="4">LISTA DE COTEJO EP 1-1</td></tr>
<tr><td colspan="4">ALUMNO(A):</td></tr>
<tr><td colspan="2">PLANTEL: MIGUEL HIDALGO
SEMESTRE: CUARTO</td><td colspan="2">EVIDENCIA: EJERCICIO DE EVENTOS</td></tr>
<tr><td colspan="2">CARRERA: PROGRAMACIÓN</td><td colspan="2">ENTE EVALUADOR: HETEROEVALUACIÓN</td></tr>
<tr><td colspan="2">SEGUNDO PARCIAL</td><td colspan="2">TIPO DE EVALUACIÓN: FORMATIVA, SUMATIVA</td></tr>
<tr><td colspan="4">COMPETENCIA PROFESIONAL: 1. Utiliza JavaScript para el manejo de eventos</td></tr>
<tr><td colspan="4">COMPETENCIA DISCIPLINAR:
C12 Utiliza las tecnologías de la información y comunicación para investigar, resolver problemas, producir materiales y transmitir información.</td></tr>
<tr><td colspan="4">COMPETENCIA GENÉRICA: 4. Escucha e interpreta y emite mensaje pertinentes en distintos contextos mediante la utilización de medios, códigos y herramientas apropiadas.</td></tr>
<tr><td rowspan="2">CRITERIOS A EVALUAR</td><td colspan="2">PUNTOS</td><td rowspan="2">OBSERVACIONES</td></tr>
<tr><td>VALOR</td><td>LOGRADO</td></tr>
<tr><td colspan="4">Los ejercicios contienen:</td></tr>
<tr><td>1. Manejadores de eventos.
2. Métodos de eventos.
3. El uso de eventos en un formulario.
4. La cantidad de ejercicios solicitada por el docente.
5. El desarrollo correcto de los procedimientos para lograr el objetivo.
6. Entrega en tiempo y forma.</td><td>5
5
5
5
5
5</td><td></td><td></td></tr>
<tr><td>TOTAL</td><td>30</td><td></td><td></td></tr>
<tr><td>RESULTADO DE EVALUACIÓN:</td><td></td><td rowspan="2"></td><td rowspan="2"></td></tr>
<tr><td>FECHA DE APLICACIÓN:</td><td></td></tr>
</table>

RESPONSABLE DE LA EVALUACIÓN

Estimado alumno(a), ahora TÚ ERES:

